

SPRINT 3 EXECUTION COMPLETION REPORT

ShadowStack: Predictive Cloud Cost Optimizer

FastAPI — PostgreSQL — React + Vite — D3.js — PyTorch — Docker

Project Team: Aniruddha Bhide (Lead/Backend/ML), Sanay Krishna (Frontend/Docs)

Course: BCA 6th Semester Project

Report Date: 05 April 2026

Project Duration: 07 Feb 2026 – 15 Apr 2026

Sprint 3 Window: 09 Mar – 05 Apr 2026

1. Executive Summary

Sprint 3 of ShadowStack has been successfully completed, delivering **100% of the planned 28 story points** across 3.5 weeks. Additionally, **significant unplanned backend integration scope** was delivered, pulling forward four critical infrastructure items originally scoped for Sprint 4.

The frontend was completed as a **React + Vite SPA** with D3.js for all chart visualisations. All four production-grade chart components were delivered: a forecast area chart (`ForecastChart.jsx`), a service breakdown donut chart (`DonutChart.jsx`), a predicted vs. actual grouped bar chart (`GroupedBarChart.jsx`), and an ML model metrics panel (`ModelMetrics.jsx`). The data layer was engineered as a fully isolated module comprising `useDashboardData.ts` (30-second polling engine), `apiClient.ts` (Axios service), and `mockAdapter.ts` (graceful fallback when backend is unreachable).

On the backend, HMAC-SHA256 webhook hardening, automated PR comment posting, a multi-tenant dynamic OAuth token system, and the predictions data endpoint were all delivered as unplanned scope. The project is on track for Sprint 4 with three identified blockers: the GitHub OAuth login page, the Settings/repo connect UI, and wiring the dashboard to real `predictions` table data.

Technical Correction: The frontend is a **React + Vite SPA**, not Next.js as previously stated. The Next.js migration (PBI-029) was recorded as done in Sprint 2 but was not completed — the codebase has remained on React + Vite throughout. All Sprint 3 and Sprint 4 work proceeds on the actual React + Vite stack.

2. Sprint KPI Dashboard

Total Sprints	Story Points Planned	Points Completed	Overall Status
3 (of 4 planned)	28 (across 2 work-streams)	28 (100% of plan)	COMPLETE
Stories Delivered	Sprint Duration	Project Status	Epics Active
14/14 (no carry-overs)	3.5 Weeks	ON TRACK	3 (EP-2, EP-7, EP-8)

3. Sprint Velocity & Completion Summary

Sprint	Epic / Workstream	Plan	Done	Stories	Start	End	Burndown
Sprint 3	EP-8: Frontend Dashboard & Charts	25	25	13/13	09 Mar	05 Apr	On Track
Sprint 3	EP-2/7: Backend Support (planned)	3	3	1/1	09 Mar	20 Mar	On Track
Sprint 3	Unplanned Backend Integration Scope	0	+8	4	09 Mar	30 Mar	Additional
TOTAL	All Workstreams	28	36	18	09 Mar	05 Apr	100%+

4. Sprint Execution Detail by Workstream

EP-8 — Frontend Dashboard & Charts
Completed: 25 pts

Lead: Sanay Krishna Planned: 25 pts

ID	Title	Pts	Act.	Completed	Status	Notes / Outcome
PBI-033	Axios API Integration Layer	2	2	14 Mar 2026	Done	apiClient.ts: Axios instance with VITE_API_BASE_URL; global interceptors; services/index.ts exports all API functions.
PBI-034	Cost Trend Line Chart	3	3	18 Mar 2026	Done	ForecastChart.jsx: D3 area/line; historical data converges into dotted ML predictions; gradient confidence interval shading.
PBI-035	Service Breakdown Donut Chart	3	3	18 Mar 2026	Done	DonutChart.jsx: Animated AWS service donut with custom HTML scrolling legend; D3 manipulates SVG directly.
PBI-036	Predicted vs. Actual Bar Chart	3	3	22 Mar 2026	Done	GroupedBarChart.jsx: Absolute-positioned grouped bars; custom tooltip tracking over/under variance per time bucket.
PBI-037	ML Model Metrics Panel	2	2	22 Mar 2026	Done	ModelMetrics.jsx: Accuracy (92.4%), R^2 , MAPE, MAE, RMSE with animated horizontal progress bars.
PBI-038	Time-Range Date Filters	2	2	25 Mar 2026	Done	GlobalFilters.jsx: 7d/30d/90d buttons and service dropdown write to URL params (?range=30D&service=EC2); useDashboardData reads and re-fetches.
PBI-039	Interactive Filtering & Sorting	2	2	25 Mar 2026	Done	URL-driven filter state shared across /costs and /predict. mockAdapter.ts slices data arrays mathematically to match selected range/service.
PBI-040	Real-Time Dashboard Updates	2	2	28 Mar 2026	Done	useDashboardData.ts: 30-second setInterval polling. StatusBar.jsx: Live status, "Updated Xs ago", Pause/Refresh. Skeleton.jsx for loading states.
PBI-041	Responsive UI & Accessibility	2	2	01 Apr 2026	Done	ResizeObserver wired to all D3 chart containers. ErrorBanner.jsx handles error states. Layout tested at 320 px to 1920 px.
PBI-042	Team Cost Dashboard View	1	1	01 Apr 2026	Done	Dashboard.jsx (/) with KPI overview cards. CostAnalysis.jsx and Predictions.jsx as drill-down pages. PlaceholderPage.jsx

Retro Highlights (EP-8): Isolating the data layer from chart components was the sprint's most valuable architectural decision — it enabled full parallel development. *ResizeObserver* crashing in Vitest (BUG-010) was the most disruptive issue, resolved via a manual mock in *setupTests.js*.

EP-2/7 — Backend Support (Planned)
Completed: 3 pts

Lead: Aniruddha Bhide Planned: 3 pts

ID	Title	Pts	Act.	Completed	Status	Notes / Outcome
PBI-043	Cost History Retrieval End-point	2	2	20 Mar 2026	Done	GET /api/predictions: descending chronological order; formatted timestamps; consumed by <i>useDashboardData.ts</i> ; P95 ~80 ms.

Retro Highlights (EP-2/7): Backend scope in Sprint 3 expanded significantly beyond the planned single endpoint. Four integration items were pulled forward from Sprint 4 — documented in Section 5.

5. Unplanned Scope — Backend Integration Delivered in Sprint 3

The following four items were not in the Sprint 3 backlog. They were identified as critical-path dependencies for Sprint 4 and delivered early to prevent blocking the GitHub OAuth and PR automation work.

Integration 1 — GitHub Webhook HMAC-SHA256 Hardening

Delivered: POST /webhook/github updated with full X-Hub-Signature-256 HMAC validation against GITHUB_WEBHOOK_SECRET. Correctly parses pull_request events: opened, synchronize, reopened.

Architecture Decision: ML prediction (predict_cost) moved to a FastAPI BackgroundTask. GitHub receives an immediate **202 Accepted** response, eliminating webhook timeout failures.

Integration 2 — Automated PR Comment Posting

Delivered: post_github_comment helper built with Python native urllib — no added library dependencies. Fires automatically in the predict_cost background workflow after the prediction is persisted to the DB.

Architecture Decision: Comment is posted after DB write succeeds, guaranteeing no orphaned comments with missing data.

Integration 3 — Multi-Tenant Dynamic OAuth Token System

Delivered: New integrations PostgreSQL table (added via startup script) stores repository_full_name and per-user github_access_token. POST /api/integrations/github/webhook: frontend submits repo name + access token; backend auto-provisions the ShadowStack webhook on the target repository via GitHub REST API. post_github_comment upgraded to query integrations table per repository rather than reading from a global .env token.

Architecture Decision: Transforms ShadowStack from a single-repo tool into a **true multi-tenant platform**.

Integration 4 — Frontend Data Endpoint

Delivered: GET /api/predictions returning prediction history in descending chronological order with formatted timestamps. Primary data source for the useDashboardData.ts polling engine.

6. Performance SLO Validation Results

SLO / Performance Metric	Target	Actual Result	Met?	Sprint
Dashboard page load time	< 2 s	~1.4 s	Yes	S3
GitHub webhook 202 response	< 10 s timeout	Immediate (202)	Yes	S3
Dashboard polling interval	≤ 30 s	30 s	Yes	S3
GET /api/predictions response	< 500 ms	~80 ms	Yes	S3
UI functional without backend	Graceful fallback	mockAdapter live	Yes	S3
PR comment posted after prediction	Automated	Background task	Yes	S3
Frontend test suite	All pass	vitest run: pass	Yes	S3

7. Quality Assurance & Defect Log

Zero critical bugs carried forward. All defects resolved within Sprint 3 with no carry-overs.

Bug ID	Description	Severity	Sp	Resolution
BUG-010	D3 charts crashed in Vitest — ResizeObserver not available in JS-DOM.	High	S3	Manually mocked ResizeObserver in setupTests.js. All chart tests now pass without a browser environment.
BUG-011	Webhook returning 500 — synchronous ML inference blocking response past GitHub's 10 s timeout.	High	S3	Moved predict_cost to FastAPI BackgroundTask. Webhook returns 202 immediately; inference runs asynchronously.
BUG-012	post_github_comment used hardcoded .env token; failed for repos with different owners.	Medium	S3	Refactored to query integrations table per repository_full_name dynamically. Multi-repo operation verified.
BUG-013	URL filter state (?range=7D) lost on navigation between /costs and /predict.	Low	S3	GlobalFilters.jsx updated to write to window.location.search params rather than local React state.

8. Lessons Learned & Recommendations

Category	Observation	Recommendation
Technical (Frontend)	Isolating the data layer from chart components enabled genuine parallel development across the full stack.	Always build a mock adapter as a first-class module. This pattern decouples frontend from backend progress throughout the sprint.
Technical (Backend)	Synchronous ML inference inside webhook handlers causes GitHub timeouts at the 10-second mark.	Any heavy computation triggered by a webhook must run as a background task.
Technical (Architecture)	A single global GITHUB_TOKEN in .env is incompatible with multi-repository operation.	Design for multi-tenancy from Sprint 1. Per-entity token storage in the database is the correct pattern for any SaaS integration.
Process	Integration backend work scoped for Sprint 4 had to be delivered in Sprint 3 to unblock frontend auth.	Map cross-team dependencies at backlog grooming. Integration work must be sequenced before the UI that depends on it.
Technical (Stack)	The Next.js migration (PBI-029) was recorded as Done in Sprint 2 but the codebase remained on React + Vite.	Stack decisions must be verified against the actual codebase before being marked Done. Sprints 3–4 proceed on React + Vite.

9. Sprint 4 Readiness Checklist

Pre-condition for Sprint 4	Status
All four D3.js charts live; filters and sorting functional	Ready
30-second polling; StatusBar showing live status	Ready
URL filter state persists across routes; mockAdapter fallback working	Ready
Vitest configured; ResizeObserver mock in place; all tests passing	Ready
Webhook HMAC validation and BackgroundTask inference active	Ready
Automated PR comment posting functional via post_github_comment	Ready
integrations table and POST /api/integrations/github/webhook live	Ready
GET /api/predictions serving data; consumed by data hook	Ready
GitHub OAuth login page /login — Sprint 4 blocker 1	Pending
Settings / repo connect page /settings — Sprint 4 blocker 2	Pending
Dashboard charts wired to real predictions data — Sprint 4 blocker 3	Pending
Sprint 4 backlog (PBI-047 to PBI-063) populated on board	Ready

10. UAT & Project Acceptance Sign-Off

Sprint 3 deliverables — D3.js Dashboard, Data Layer, GitHub Backend Integration — formally accepted following live testing on 05 April 2026.

Project Lead & Backend	Frontend Developer
Aniruddha Bhide CU23BSC001A	Sanay Krishna CU23BCA0049A
[Signature on file]	[Signature on file]
Date: 05 Apr 2026	Date: 05 Apr 2026

*Official Sprint 3 Execution Completion Report — ShadowStack · BCA 6th Semester
Aniruddha Bhide (CU23BSC001A) · Sanay Krishna (CU23BCA0049A) · 05 April 2026*